

General idea

In this task I implemented a fake data generator using MySQL stored procedures.

The main idea is to generate user data directly in MySQL instead of using external libraries.

The data is not truly random. It depends on input values (seed, batch, locale), so if you run the same query again, you'll get the same result.

Main procedure

```
generate_batch(seed_val, batch_val, locale_val)
```

This procedure generates 10 users and returns them as a table.

Parameters

seed_val – used for generating values

batch_val – used to get different groups of users

locale_val – selects names for a specific region (for example en_US)

Example:

```
CALL generate_batch(1, 0, 'en_US');
```

Tables

There are several lookup tables:

first_names

last_names

streets

cities

All tables use a locale column, so different regions can be stored together instead of creating separate tables.

How generation works

Random numbers

I didn't use random() function. Instead I used:

`CRC32(CONCAT(...))` (even knowing it's not perfect for randomness)

This generates a number based on input values.
Because of that, the same inputs always produce the same outputs.

Names

To select a name:

*count how many names exist for that locale
generate a number
use modulo to get an index
select name using LIMIT and OFFSET*

Full name is just combining (concat) first name and last name.

Regarding variations, I didn't rely only on large datasets.
Instead I combined smaller lookup tables with deterministic variations
(seed, batch, index, titles, etc.), which allows generating many unique users efficiently.

Titles (variation)

Depending on locale:

en_US → Mr., Ms.
de_DE → Herr, Frau

Sometimes title is added, sometimes not(randomly).

Email

Email is generated from the name, like:

`firstname.lastname@example.com`

Everything is converted to lowercase.

Phone

Phone numbers are generated using parts of the number from CRC32.

Formats:

en_US → +1-XXX-XXX-XXXX
de_DE → +49-XXX-XXXXXXX

It's not a real phone number, just formatted(for realistic look).

Address

Address is generated using streets and cities tables with locale support.

Steps:

generate house number
pick street based on locale
pick city based on locale

Example:

12 Main St, New York

or

5 Bahnhofstrasse, Munich

Coordinates

Coordinates are generated using two values between 0 and 1.

Then they are converted using formulas:

$latitude = asin(2u - 1)$
 $longitude = 2\pi v - \pi$

This avoids incorrect distribution (for example too many points near poles)
and gives a more uniform distribution on the sphere.

Physical attributes

Height is generated using deterministic values in a range (150–200 cm).

It's not a perfect distribution, but it gives enough vars between users.

Reproducibility

If I run:

```
CALL generate_batch(1, 0, 'en_US');
```

multiple times, I always get the same users.

If I change seed or batch, I get different data.

Batch works like pagination:

batch 0 → first 10 users

batch 1 → next 10 users

Notes

duplicates can happen (due to limited name sets)

more names → better variation

more fields can be added later (address improvements, more locales, etc.)

Benchmark

Benchmark was executed by generating multiple batches.

Result (locally):

Total users: 1000

Time: ~0.13 sec

Speed: ~7400 users/sec

Result (deployed)

Total users: 100

Time: 2.93 sec

Speed: 34.08 users/sec

As we can see, after deploying whole task, rendering benchmarks is slowing down, but still working. Difference between local call and published version is really high.

Additional note

Random generation is separated from formatting logic, and deterministic approach allows reproducible results based on seed, batch and index.